

DAY-19 — Logs & Monitoring

CyberNotes • Detailed Beginner Reference (Improved Edition)

Learning goal: Build a practical, working understanding of **logs & monitoring**, connect the concept to real cybersecurity work, and practice it safely in a local lab.

Study rule: Understand a command or concept before copying it. Security tools are powerful because they interact with real systems — your practice environment should always be intentionally controlled and authorized.

1. Core Concepts

Core idea: Logs & Monitoring is a building block for practical cybersecurity. This section explains what each term actually means, where you'll encounter it, and how it connects to the bigger picture.

Why it matters: Security work depends on understanding normal system and network behavior before you can recognize abnormal behavior. This topic connects directly to Linux, networking, web security, monitoring, and troubleshooting — the threads running through this whole 20-day series.

Vocabulary covered: log sources, timestamps, levels, authentication logs, web logs, grep, correlation, alerting.

Reference	What to remember
<code>journalctl</code>	systemd journal
<code>grep</code>	search text logs
<code>tail -f</code>	follow appended log data
timestamp	event timing
source IP	network origin field
status code	web response result

2. Concept Explanations

Each term below is explained in plain language, with a note on why it matters for security work. Read these before running any commands.

1. log sources

Different components of a system generate their own logs: the OS (via the systemd journal or `/var/log`), authentication (`/var/log/auth.log`), and applications like web servers each write independently. Knowing where a given event would be logged is the first step of any investigation — the right log rarely just falls into your lap.

2. timestamps

Every log entry should carry a timestamp marking exactly when the event occurred, which is what lets you reconstruct a sequence of events across multiple log sources. A common real-world complication is time

zone or clock-sync mismatches between systems, which can make events line up incorrectly unless you account for it.

3. levels

Log levels (DEBUG, INFO, WARNING, ERROR, CRITICAL) indicate the severity or importance of an entry, letting you filter out routine noise and focus on what actually needs attention. Setting the right level is a balance — too verbose and important events get buried, too quiet and you lose visibility.

4. authentication logs

Authentication logs (`/var/log/auth.log` on Debian-based systems, or the journal on others) record login attempts, both successful and failed, along with the source. Reviewing failed login patterns is one of the most direct ways to spot brute-force attempts or unauthorized access attempts.

5. web logs

Web server logs record every request handled — source IP, timestamp, requested path, and status code — forming a detailed record of exactly how a web application has been used. Unusual patterns here, like a flood of 404s probing different paths, often indicate automated scanning or reconnaissance.

6. grep

As covered on Day 3, `grep` is the workhorse for pulling specific events out of large log files, e.g. `grep -i failed /var/log/auth.log` isolates failed login attempts from everything else in the file.

7. correlation

Correlation means linking related events across different logs or systems to build a full picture — for example, matching a suspicious login in the auth log to an unusual outbound connection in the network logs around the same timestamp. A single log entry in isolation rarely tells the whole story.

8. alerting

Alerting automatically notifies someone when a defined condition is met — a threshold of failed logins, a specific error pattern — instead of requiring a human to manually watch logs all day. Well-tuned alerts catch real issues quickly; poorly tuned ones generate so much noise that important alerts get ignored, a failure mode known as alert fatigue.

3. Safe Practical Lab

Step 1. Define the lab target and the exact question you are trying to answer.

Step 2. Run the smallest safe observation or command that can answer that question.

Step 3. Read the output carefully; record important fields, errors, timestamps, and assumptions.

Step 4. Change one variable at a time and compare the result.

Step 5. Write a short conclusion and clean up temporary files or services.

Example lab sequence:

```
journalctl -n 20 2>/dev/null  
tail -n 20 /var/log/auth.log 2>/dev/null  
grep -i failed /var/log/auth.log 2>/dev/null | tail
```

4. Common Mistakes & Troubleshooting

- Running a command without checking its target, current directory, or privileges.
- Confusing a tool's output with proof of a vulnerability or compromise — output is evidence, not a conclusion.
- Ignoring error messages instead of reading the exact reason for failure.
- Changing several settings at once, making it hard to tell what actually fixed or broke the lab.
- Saving real credentials, tokens, or personal data in practice notes or public repositories.
- Testing a public target just because it's reachable — reachability is never the same as authorization.

Troubleshooting checklist:

- What system am I on?
- What exact command/request did I run?
- What did I expect?
- What happened instead?
- What does the error message say?
- Can I reproduce it in a clean local lab?
- What changed since the last successful attempt?

5. Deep-Dive Questions

Answer these in your own words after finishing the lab — they're designed to check real understanding, not memorization.

Q1. What problem does log sources solve, in your own words?

Notes: _____

Q2. What is the difference between normal behavior and a suspicious observation in this topic?

Notes: _____

Q3. Which command, log, packet, file, or response would you inspect first, and why?

Notes: _____

Q4. What could cause a misleading result or false positive here?

Notes: _____

Q5. How would you document your observation so another learner could reproduce it safely?

Notes: _____

Q6. Which security control reduces the risk associated with this topic?

Notes: _____

6. Revision Checklist

- I can explain logs & monitoring in my own words.
- I can identify the main terms without looking at the notes.
- I can perform the example in a controlled lab.
- I can interpret the output instead of only copying it.
- I know at least one defensive or troubleshooting use for the concept.
- I can document the result clearly and safely.

7. Safety & Scope

Use these commands and tools only on systems you own, a local virtual machine, a CTF, an intentionally vulnerable application, or a target for which you have explicit written permission.

Keep a written scope: target, allowed actions, time window, and stop conditions. Never scan or test random public systems.

When practicing, save your notes and outputs so you can explain what you observed rather than blindly running commands.

Next step: After Day-19, review your notes and connect this topic to the next subject in the series. The goal is not to memorize commands — it's to understand systems well enough to reason about security.

8. Quick Reference & Personal Notes

Topic: Day-19 — Logs & Monitoring

Three things I learned:

1. _____
1. _____
1. _____

Commands / concepts to memorize:

- `journalctl` — systemd journal
- `grep` — search text logs
- `tail -f` — follow appended log data
- `timestamp` — event timing
- `source IP` — network origin field
- `status code` — web response result

My lab result

What I tested: _____

What I observed: _____

What I would investigate next: _____

Final reminder: A strong cybersecurity learner can explain both the offensive concept and the defensive reason for learning it. Keep practice authorized, reproducible, and documented.